SMART RESTAURANT MANAGEMENT SYSTEM

API Specification

REST endpoints and real-time events

DOCUMENT ID	SRMS-08
VERSION	1.0
DATE	06 November 2026
STATUS	Draft for review
PHASE	Phase 1: event-driven core | Phase 2: AI camera detection
PURPOSE	Contract between the clients and the backend: endpoints, roles, events and sample payloads.

1. Conventions

- Base URL: `/api/v1`. JSON request and response. UTF-8. Timestamps in ISO 8601 UTC.
- Auth types: **Guest** (session token from QR scan), **Staff** (JWT with role), **Service** (signed token for AI service and payment webhooks), **Public** (none).
- Errors use a standard body: `{ "error": { "code": "TABLE_NOT_AVAILABLE", "message": "..." } }` with proper HTTP status.
- Lists are paginated with `?page=&limit=`. State-changing payment endpoints require an `Idempotency-Key` header.
- All amounts are in paise or fixed decimals as strings to avoid rounding errors (final choice recorded in the API contract).

2. REST Endpoints

2.1 Session and Menu (Guest)

Method	Path	Auth	Description
POST	/sessions/scan	Public	Body: table token. Creates or joins the active session; returns session token and table info
GET	/sessions/current	Guest	Current session, table label, guest count
PATCH	/sessions/current	Guest	Set guest count or phone
GET	/menu	Guest	Categories, items, variants, modifiers, availability
GET	/menu/items/:id	Guest	Item detail

2.2 Cart, Orders and Alerts (Guest)

Method	Path	Auth	Description
GET	/cart	Guest	Shared cart for the session
PUT	/cart/items	Guest	Add, update or remove item with variant, modifiers, note
POST	/orders	Guest	Place order from the cart; server recalculates prices
GET	/orders	Guest	Orders and live item status for the session
POST	/orders/items/:id/cancel	Guest	Cancel if still Pending or Accepted
POST	/alerts/call-waiter	Guest	Raise Call Waiter alert
POST	/alerts/request-bill	Guest	Raise Request Bill alert and set Payment Pending

2.3 Billing and Payments

Method	Path	Auth	Description
GET	/bill	Guest	Running bill with tax breakdown
POST	/payments	Guest	Create payment (method, amount); returns gateway order. Needs Idempotency-Key
GET	/payments/:id	Guest	Payment status (server truth)
POST	/webhooks/razorpay	Service	Gateway webhook; signature verified
POST	/payments/cash	Staff (waiter+)	Record cash payment for the bill
POST	/payments/:id/refund	Manager+	Refund with reason; owner above limit
POST	/bills/:id/discount	Manager+	Apply discount with reason (audited)
GET	/bills/:id/invoice	Guest / Staff	PDF invoice or receipt link

2.4 Staff: Tables, Floor Map and Sessions

Method	Path	Auth	Description
GET	/floor	Staff	Snapshot: floors, tables, status, guests, timers, open alert counts
GET	/tables/:id	Staff	Table detail with session, orders, bill, alerts
POST	/tables/:id/seat	Waiter+	Seat walk-in; create session with guest count
POST	/tables/:id/status	Manager+	Override status; body: status, reason
POST	/sessions/:id/close	Manager+	Close session with reason (PAID, WALK_OUT, MANAGER)
GET	/tables/:id/history	Manager+	Status log
GET	/dashboard/summary	Manager+	Occupancy, capacity %, average dwell, open alerts

2.5 Staff: Orders and Kitchen

Method	Path	Auth	Description
GET	/kitchen/tickets	Kitchen+	Open items ordered by time
PATCH	/order-items/:id/status	Kitchen / Waiter	Accept, Preparing, Ready (kitchen); Served (waiter)
POST	/staff/orders	Waiter+	Place order on behalf of a table
PATCH	/order-items/:id	Waiter+	Edit quantity or note; cancel with reason
PATCH	/menu/items/:id/availability	Kitchen+	Mark sold out or available

2.6 Alerts and Cleaning

Method	Path	Auth	Description
GET	/alerts	Staff	Open alerts (filter by table, type, assignee)
POST	/alerts/:id/acknowledge	Waiter+	Acknowledge
POST	/alerts/:id/resolve	Waiter+	Resolve
GET	/cleaning/tasks	Cleaner+	Queue ordered by priority and age
POST	/cleaning/tasks/:id/start	Cleaner+	Start; table becomes Cleaning In Progress
POST	/cleaning/tasks/:id/done	Cleaner+	Complete; table becomes Available
POST	/tables/:id/out-of-service	Cleaner+	Mark out of service with note

2.7 Admin

Method	Path	Auth	Description
POST/PUT/DELETE	/admin/menu/categories, /admin/menu/items	Manager+	Menu CRUD, image upload, CSV import
POST/PUT	/admin/floors, /admin/tables	Owner	Floors, tables and layout positions
POST	/admin/tables/:id/qr/rotate	Owner	Rotate QR token
GET	/admin/qr/sheet	Owner	Printable PDF of all table QR codes
CRUD	/admin/staff	Owner	Staff accounts and roles
PUT	/admin/settings	Owner	Taxes, thresholds, timeouts, gateway keys
POST	/auth/login, /auth/pin-login, /auth/refresh	Public	Staff authentication

2.8 Reports

Method	Path	Auth	Description
GET	/reports/sales	Manager+	Sales by period, category, item, payment method
GET	/reports/tables	Manager+	Turnover, dwell, cleaning time
GET	/reports/service	Manager+	Alert response, prep time
GET	/reports/export	Manager+	CSV or PDF export

2.9 Phase 2: Vision

Method	Path	Auth	Description
POST	/vision/events	Service	Batch of detection events with confidence, camera, zone
GET	/vision/suggestions	Manager+	Pending low-confidence suggestions
POST	/vision/suggestions/:id/confirm	Waiter+	Apply suggestion via state engine
POST	/vision/suggestions/:id/dismiss	Waiter+	Dismiss suggestion
CRUD	/admin/cameras, /admin/cameras/:id/zones	Owner	Cameras and table zones

3. WebSocket Events

Namespace `/live`. Staff join `restaurant:{id}`; guests join `session:{id}`.

Event	Audience	Payload (summary)
table.updated	Staff	tableId, status, guestCount, statusChangedAt, alertCount
session.opened / session.closed	Staff	sessionId, tableId, reason
order.placed	Kitchen, Waiter, Manager	orderId, tableLabel, items
item.status	Guest, Staff	orderItemId, status, timestamps
alert.raised / alert.updated	Staff	alertId, tableId, type, severity, status, assignee
cleaning.task	Cleaner, Manager	taskId, tableId, status, priority
bill.updated	Guest, Waiter	total, amountPaid, status
payment.status	Guest, Waiter	paymentId, status
menu.availability	Guest, Staff	itemId, isAvailable
cart.updated	Guest	cart snapshot
vision.suggestion (Phase 2)	Manager, Waiter	suggestionId, tableId, type, confidence

4. Example Payloads

```
POST /api/v1/sessions/scan
{ "tableToken": "k9X2..." }

200 OK
{ "sessionToken": "eyJ...",
  "table": { "label": "T04", "floor": "Indoor" },
  "session": { "id": "...", "startedAt": "2026-11-06T07:10:00Z" } }
```

```
POST /api/v1/tables/:id/status   (Manager)
{ "status": "AVAILABLE", "reason": "Guests left, table already clean" }

200 OK
{ "tableId": "...", "from": "NEEDS_CLEANING", "to": "AVAILABLE" }
```